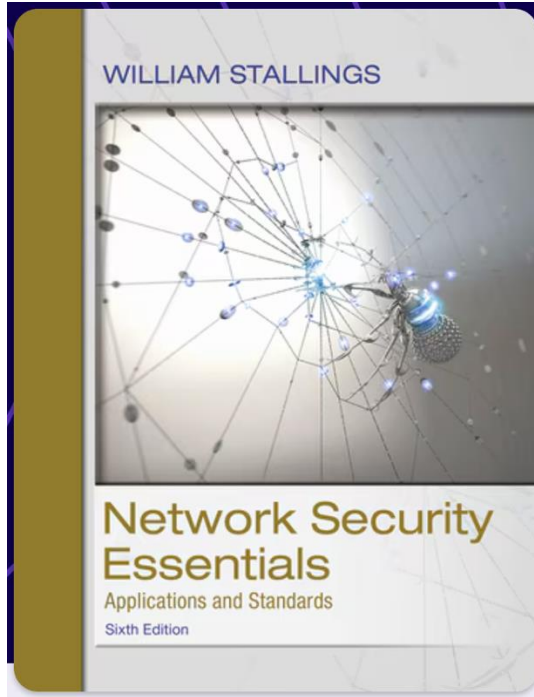


Public Key Cryptography

Dr. Yiwen Hu

Acknowledgement



You can find it on Blackboard

The content of slides is from the textbook and Dr. Hu's supplementary materials

Reminder: Please do not post course materials online or share them with others without the instructor's permission

Common Public-Key Cryptosystems

- **RSA:** One of the most widely used asymmetric algorithms. Commonly used for encrypting small pieces of data (such as symmetric keys) and for digital signatures.
- **ECC (Elliptic Curve Cryptography):** Newer than RSA. Offers similar security with much shorter keys, which makes it well suited for mobile and IoT devices.
- **Diffie-Hellman:** Not an encryption algorithm itself, but a key exchange protocol that uses asymmetric math to let two parties agree on a shared secret key.

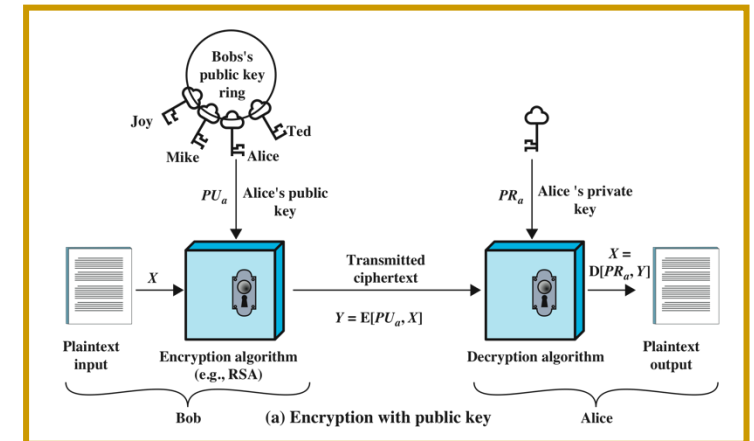
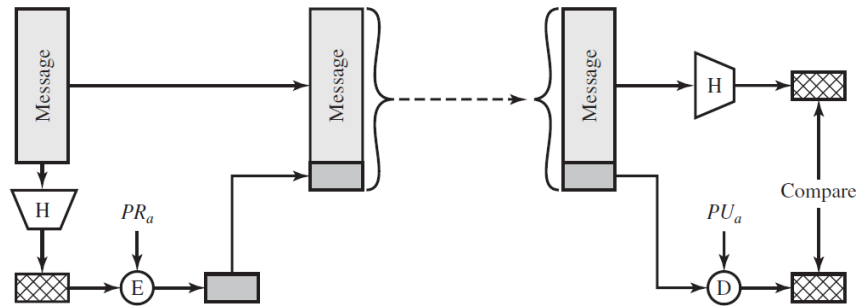
Common Public-Key Cryptosystems

RSA: One of the most widely used asymmetric algorithms.

RSA is used whenever someone, like a server or a user, needs a public/private key pair

Commonly used for

- encrypting small pieces of data (such as symmetric keys)
- digital signatures



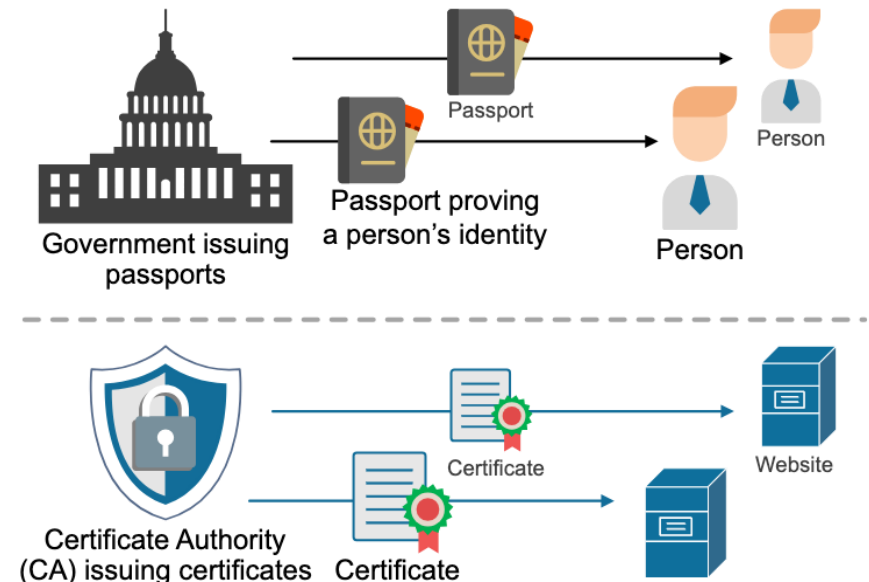
Attacking Scenario

- If I'm an attacker. I follow the RSA steps and create my own key pair. Then I post the public key online and tell everyone, "This is Bob's public key. From now on, if you want to send Bob a secret, encrypt it with this key."
- The same thing could happen if someone pretends to be a Google server or a UMBC server and publishes a so-called public key online

The gap here: how do we know that the person publishing a public key really is who they claim to be?

Recap -What is Certificate Authority (CA)

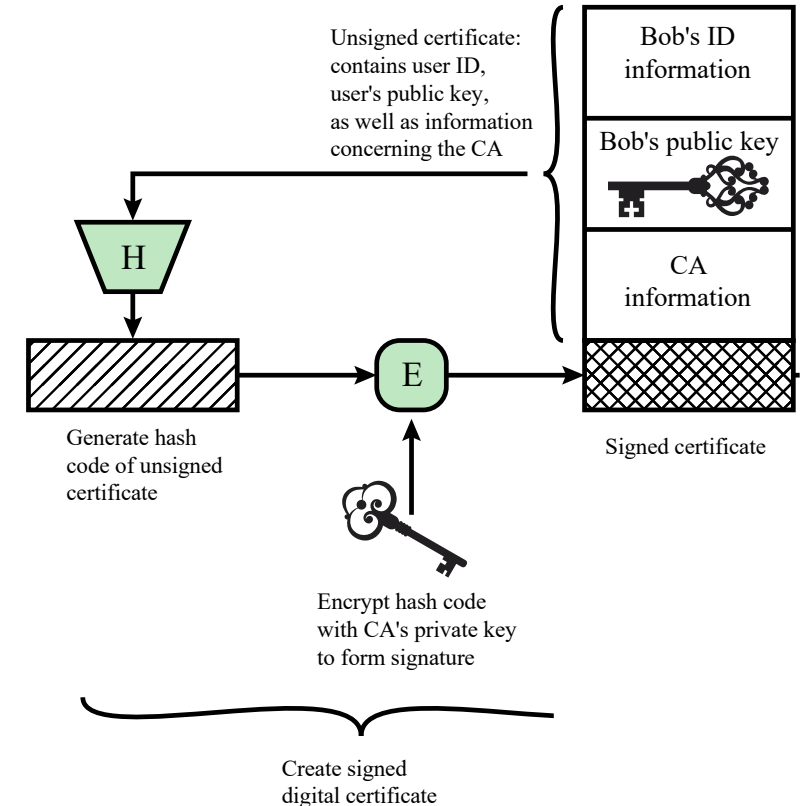
- A **Certificate Authority** is a trusted third party that vouches for the **binding** between a **public key** and an **identity**
 - **The problem:** anyone can generate a key pair and publish the public key claiming "this is Bob's public key." Nothing in the key itself tells you whether it was really generated by Bob, or by an attacker.
 - We need a trusted third party (**a CA**), like a passport office, to verify and vouch: *"this really is Bob's public key."*



Recap -What a CA does

After an entity (e.g., Bob) creates a key pair (e.g., using RSA), it sends its **public key** and identity information to a CA. This request is called a **Certificate Signing Request (CSR)**. **The private key never leaves the entity.**

- The CA verifies that the applicant really matches the claimed identity (e.g., Bob)
- The CA issues a certificate: a document containing the identity (e.g., Bob ID), the public key (e.g., Bob's public key), validity period, and issuer info (CA info)
- The CA signs the certificate with the CA's own private key



Who can be a CA?

- **Who can be a CA?** Technically, **anyone**. One OpenSSL command gives you a root key, and you can start signing certificates for anybody.

Demo: I generate a brand-new key pair on my laptop, label it "www.google.com", and sign it with my own CA

```
Last login: Wed Sep 23 00:43:45 on ttys000
[yiwen@Mac ~ % mkdir -p ~/Desktop/ca-demo && cd ~/Desktop/ca-demo
[yiwen@Mac ca-demo %
[yiwen@Mac ca-demo %
[yiwen@Mac ca-demo % openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
    -keyout myCA.key -out myCA.crt \
    -subj "/CN=Dr Hu's Totally Legit Root CA"
Generating a 2048 bit RSA private key
.....+++++
.....+++++
writing new private key to 'myCA.key'
-----
[yiwen@Mac ca-demo %
[yiwen@Mac ca-demo %
[yiwen@Mac ca-demo % openssl req -newkey rsa:2048 -nodes \
    -keyout google.key -out google.csr \
    -subj "/CN=www.google.com"
Generating a 2048 bit RSA private key
.....+++++
.....+++++
writing new private key to 'google.key'
-----
[yiwen@Mac ca-demo %
[yiwen@Mac ca-demo %
[yiwen@Mac ca-demo % openssl x509 -req -in google.csr -CA myCA.crt -CAkey myCA.key \
    -CAcreateserial -days 365 -out google.crt
Signature ok
subject=/CN=www.google.com
Getting CA Private Key
[yiwen@Mac ca-demo %
[yiwen@Mac ca-demo %
[yiwen@Mac ca-demo % openssl x509 -in google.crt -noout -subject -issuer
subject= /CN=www.google.com
issuer= /CN=Dr Hu's Totally Legit Root CA
[yiwen@Mac ca-demo %
[yiwen@Mac ca-demo % █
```

I become a CA

I generate a new key and label it "Google"

I, the CA, vouch for the lie

Attack scenario

```
Last login: Wed Sep 23 00:43:45 on ttys000
yiwen@Mac ~ % mkdir -p ~/Desktop/ca-demo && cd ~/Desktop/ca-demo
yiwen@Mac ca-demo %
yiwen@Mac ca-demo %
yiwen@Mac ca-demo % openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
-keyout myCA.key -out myCA.crt \
```

I become a CA

```
[ -subj "/CN=Dr Hu's Totally Legit Root CA"
Generating a 2048 bit RSA private key
.....+++++
.....+++++
writing new private key to 'myCA.key'
-----
```

```
yiwen@Mac ca-demo %
yiwen@Mac ca-demo %
yiwen@Mac ca-demo % openssl req -newkey rsa:2048 -nodes \
-keyout google.key -out google.csr \
[ -subj "/CN=www.google.com"
Generating a 2048 bit RSA private key
.....+++++
.....+++++
writing new private key to 'google.key'
-----
```

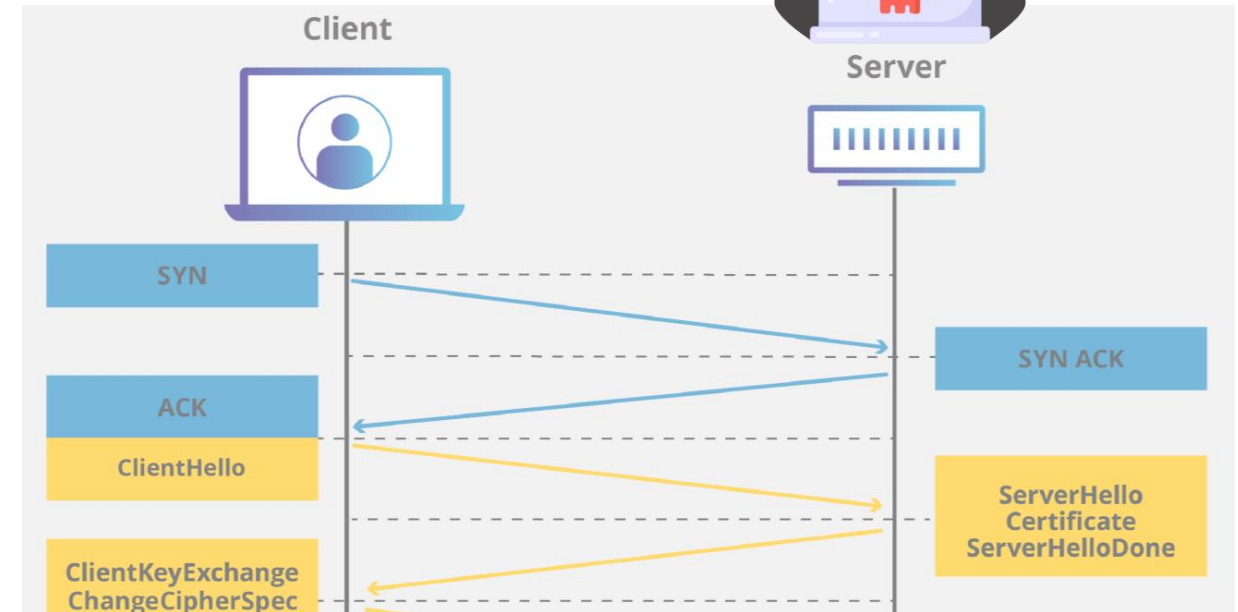
I generate a **new** key and label it "Google"

```
yiwen@Mac ca-demo %
yiwen@Mac ca-demo %
yiwen@Mac ca-demo % openssl x509 -req -in google.csr -CA myCA.crt -CAkey myCA.key \
-CAcreateserial -days 365 -out google.crt
Signature ok
```

I, the CA, vouch for the lie

```
[subject=/CN=www.google.com
Getting CA Private Key
yiwen@Mac ca-demo %
yiwen@Mac ca-demo %
yiwen@Mac ca-demo % openssl x509 -in google.crt -noout -subject -issuer
[subject= /CN=www.google.com
[issuer= /CN=Dr Hu's Totally Legit Root CA
yiwen@Mac ca-demo %
yiwen@Mac ca-demo % █
```

The attacker



An attacker deploys a malicious server, claims to be Google, and sends me a certificate for `www.google.com` — containing a key *they* generated, signed by a CA *they* created

What actually happens? Your browser catches this attack.



Your connection is not private

Attackers might be trying to steal your information from **192.168.1.199** (for example, passwords, messages, or credit cards). `NET::ERR_CERT_AUTHORITY_INVALID`

☐ Automatically report details of possible security incidents to Google. [Privacy policy](#)

HIDE ADVANCED

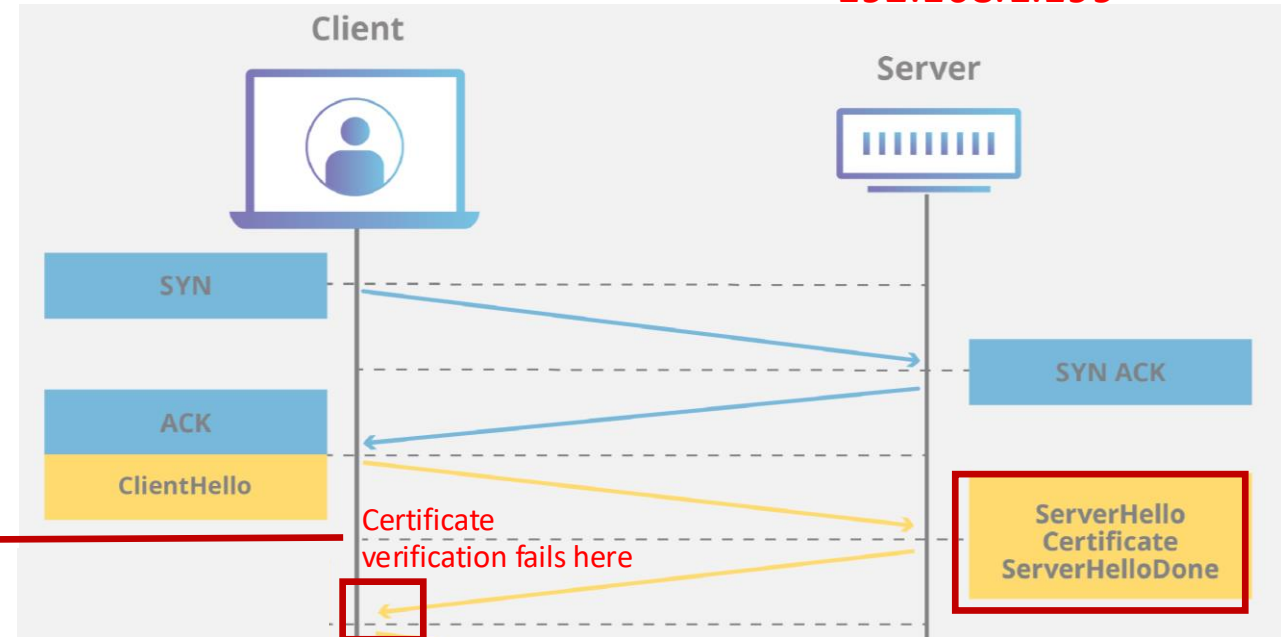
[Back to safety](#)

This server could not prove that it is **192.168.1.199**; its security certificate is not trusted by your computer's operating system. This may be caused by a misconfiguration or an attacker intercepting your connection.

[Proceed to 192.168.1.199 \(unsafe\)](#)

Your laptop (e.g.,
using chrome)

Attacker Server IP:
192.168.1.199



Your laptop Chrome reads the issuer (the CA) from Certificate → not found in the root store → refuses to continue and shows this warning.
`NET::ERR_CERT_AUTHORITY_INVALID` = "I don't know who vouched for this key."

Being a CA ≠ being a trusted CA

- Anyone can *create* a CA (I just did). But your browser only *trusts* a CA whose public key is already on your computer.

Root store = a folder on your laptop holding the public keys of the CAs your OS / browser trusts. Think of it as a **whitelist**.

- It ships with the OS / browser — you never added anything to it
- ~100+ root CAs are on the list by default
- Who decides what goes in? Whoever made your software: **Apple** (macOS/iOS), **Microsoft** (Windows), **Google** (Chrome/Android), **Mozilla** (Firefox)

Examples

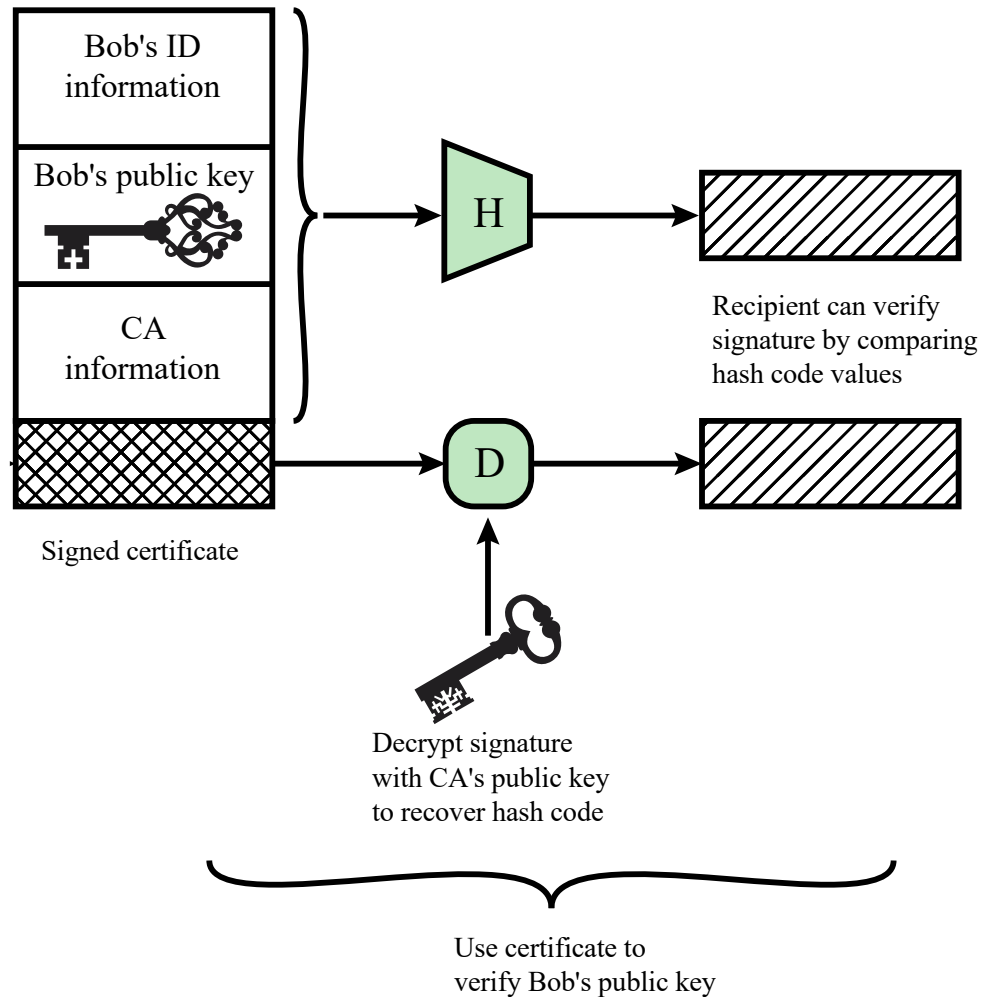
- Public CAs: DigiCert, Sectigo, GlobalSign, Let's Encrypt (free, non-profit)
- Tech companies: Google Trust Services, Apple, Microsoft, Amazon

```
security find-certificate -a /System/Library/Keychains/SystemRootCertificates.keychain | grep '"labl"' |  
sed 's/.*=="//;s/"$//' | sort
```

This command lists the names of all root CA certificates pre-installed in macOS's system trust store

```
[yiwen@Mac ~ % security find-certificate -a /System/Library/Keychains/SystemRootCertificates.keychain |  
grep '"labl"' | sed 's/.*=="//;s/"$//' | sort  
[  "labl"<blob>=0x4E65744C6F636B204172616E792028436C61737320476F6C64292046C59174616E  
A16E79  "NetLock Arany (Class Gold) F\305\221tan\303\272s\303\255tv\303\241ny  
AAA Certificate Services  
AC RAIZ FNMT-RCM  
ACCVRAIZ1  
Actalis Authentication Root CA  
AffirmTrust Commercial  
AffirmTrust Networking  
AffirmTrust Premium  
AffirmTrust Premium ECC  
Amazon Root CA 1  
Amazon Root CA 2  
Amazon Root CA 3  
Amazon Root CA 4  
Apple Root CA  
Apple Root CA - G2  
Apple Root CA - G3  
Apple Root Certificate Authority  
Atos TrustedRoot 2011  
Atos TrustedRoot Root CA ECC G2 2020  
Atos TrustedRoot Root CA ECC TLS 2021  
Atos TrustedRoot Root CA RSA G2 2020  
Atos TrustedRoot Root CA RSA TLS 2021  
Autoridad de Certificacion Firmaprofesional CIF A62634068  
Baltimore CyberTrust Root  
Buypass Class 2 Root CA  
Buypass Class 3 Root CA  
CA Disig Root R2  
Certainly Root E1  
Certainly Root R1  
Certigna  
certSIGN ROOT CA  
certSIGN ROOT CA G2  
Certum CA
```

Key management



Verifying a certificate:

find the issuer's public key in the root store →
decrypt the **signature** to recover the CA's hash →
hash the certificate yourself → **match?**

The certificate is untampered and really signed by that CA

so you trust its claim: *this public key belongs to Bob.*

```
openssl s_client -connect www.umbc.edu:443 -servername www.umbc.edu </dev/null 2>/dev/null |
openssl x509 -noout -text
```

This command performs a TLS handshake with www.umbc.edu, captures the certificate the server sends back, and prints its contents.

Remember the RSA slide:
the public key is the pair (e, n)

```
yiwen@Mac ~ % openssl s_client -connect www.umbc.edu:443 -servername www.umbc.edu </dev/null 2>/dev/null | openssl x509
-noout -text
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      06:75:cf:6e:42:e0:e2:97:0a:b5:5c:4e:a8:14:78:ff:42:ca
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=US, O=Let's Encrypt, CN=YR2
    Validity
      Not Before: Aug 21 12:54:15 2026 GMT
      Not After : Nov 19 12:54:14 2026 GMT
    Subject: CN=about.umbc.edu
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      RSA Public-Key: (2048 bit)
      Modulus:
        00:d4:c8:a2:1b:50:b4:c3:d2:df:35:ec:d8:65:f1:
        67:9b:ac:02:ed:84:16:58:f4:b5:e0:d4:03:d3:0e:
        20:83:07:94:0f:fd:f3:59:55:ff:cd:c7:68:41:52:
        b9:7e:18:5f:b9:5d:4a:1f:4b:93:25:71:9e:5d:eb:
        b8:df:2a:e8:f5:5c:a1:f6:c8:1a:68:a3:08:5d:ee:
        0f:03:df:34:dc:06:78:01:d4:15:ee:d0:02:db:9d:
        ce:f5:4f:4b:56:58:b8:fe:a2:76:f3:3c:b6:18:ee:
        a0:13:ed:be:1d:8f:e9:ad:e7:3c:82:ca:97:3e:de:
        95:af:ab:8c:3c:a7:61:52:42:3b:c6:3b:ed:2d:63:
        d5:cd:50:54:39:4c:d1:2d:f3:64:ca:e1:fd:f7:f9:
        0b:88:33:95:1e:35:78:d2:2a:20:5c:4c:59:6e:16:
        e0:05:11:60:7f:6a:39:c3:31:4e:9c:a2:4a:70:1e:
        9a:aa:08:0c:4b:7b:34:8c:68:42:9d:70:9e:e6:39:
        72:f3:27:2e:1a:a5:95:17:01:85:ba:5b:0b:80:4d:
        1a:34:83:91:ee:1e:6e:4e:b9:81:6e:c5:89:80:e7:
        3a:21:45:95:3c:b6:1e:87:39:7e:bc:d9:61:26:e3:
        cc:4a:be:5f:a4:0f:d9:ec:50:d0:d5:6b:4d:e2:11:
        43:eb
      Exponent: 65537 (0x10001)
  X509v3 extensions:
    X509v3 Key Usage: critical
      Digital Signature, Key Encipherment
    X509v3 Extended Key Usage:
      TLS Web Server Authentication
    X509v3 Basic Constraints: critical
      CA:FALSE
    X509v3 Subject Key Identifier:
      82:A2:6F:3D:56:4C:73:96:35:D8:22:5B:EE:7F:EB:85:37:C6:A2:78
    X509v3 Authority Key Identifier:
      keyid:40:15:2D:26:79:ED:32:20:9E:DF:9A:72:1D:D6:32:1F:81:0C:81:0C

  Authority Information Access:
    CA Issuers - URI:http://yr2.i.lencr.org/

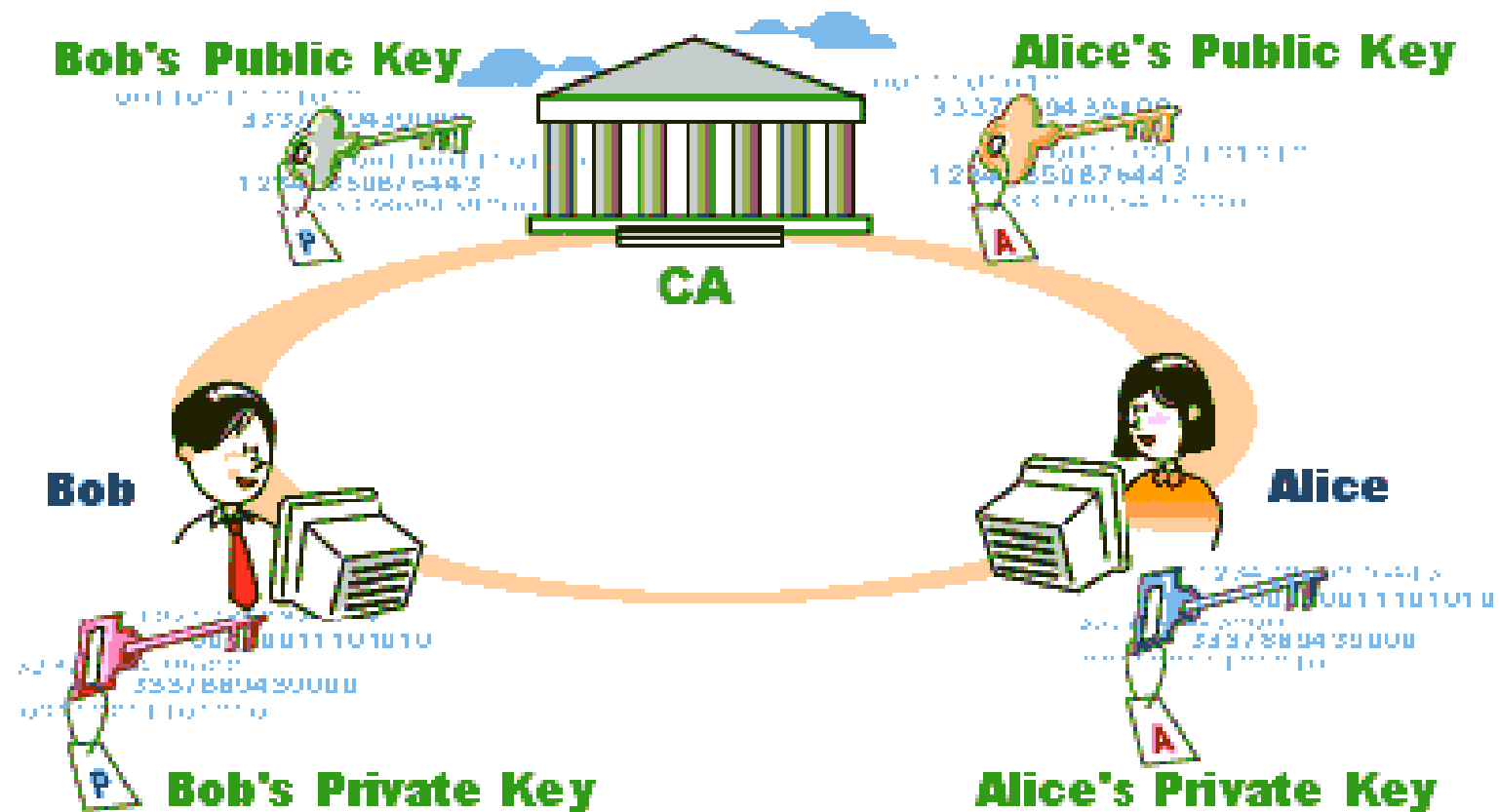
  X509v3 Subject Alternative Name:
    DNS:about.umbc.edu, DNS:alumni.umbc.edu, DNS:artsandculture.umbc.edu, DNS:getgoing.umbc.edu, DNS:giving
    .umbc.edu, DNS:globalstudent.umbc.edu, DNS:jobs.umbc.edu, DNS:magazine.umbc.edu, DNS:news.umbc.edu, DNS:options.umbc.ed
    u, DNS:planning.umbc.edu, DNS:umbc.edu, DNS:undergraduate.umbc.edu, DNS:www.umbc.edu
```

Issuer: Let's Encrypt, a CA in our root store

This is UMBC's public key

The modulus is n , 2048 bits

The exponent is e , 65537



RSA cryptosystem

- Invented by **Rivest, Shamir & Adleman** (MIT, 1977)
- Best known and widely used public-key algorithm
 - Generate a key pair: a public key (e, n) and a private key (d, n)
 - Encrypt with one key, decrypt with the other
- Theorems behind RSA
 - Modular arithmetic, Euler's totient function, Euler's product formula, Euler's theorem

The RSA Algorithm

Key Generation

Select p, q	p and q both prime, $p \neq q$
Calculate $n = p \cdot q$	
Calculate $\phi(n) = (p - 1)(q - 1)$	
Select integer e	$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$
Calculate d	$de \bmod \phi(n) = 1$
Public key	$KU = \{e, n\}$
Private key	$KR = \{d, n\}$

Greatest Common Divisor (GCD): The largest positive integer that divides two numbers evenly, with no remainder.

Example: $\gcd(12, 18) = 6$.

The divisors of 12 are 1, 2, 3, 4, 6, 12

The divisors of 18 are 1, 2, 3, 6, 9, 18

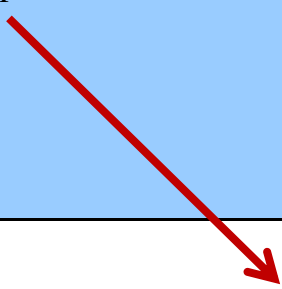
The largest one they share is 6.

If $\gcd(a, b) = 1$, then a and b are called **coprime** (relatively prime). Example: $\gcd(8, 15) = 1$.

The RSA Algorithm

Key Generation

Select p, q	p and q both prime, $p \neq q$
Calculate $n = p \cdot q$	
Calculate $\phi(n) = (p-1)(q-1)$	
Select integer e	$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$
Calculate d	$de \bmod \phi(n) = 1$
Public key	$KU = \{e, n\}$
Private key	$KR = \{d, n\}$



If $\gcd(\phi(n), e) = 1$, then $\phi(n)$ and e are called **coprime** (relatively prime).
Example: $\gcd(160, 7) = 1$.

Encryption

Plaintext:	$M < n$
Ciphertext:	$C = M^e \pmod{n}$

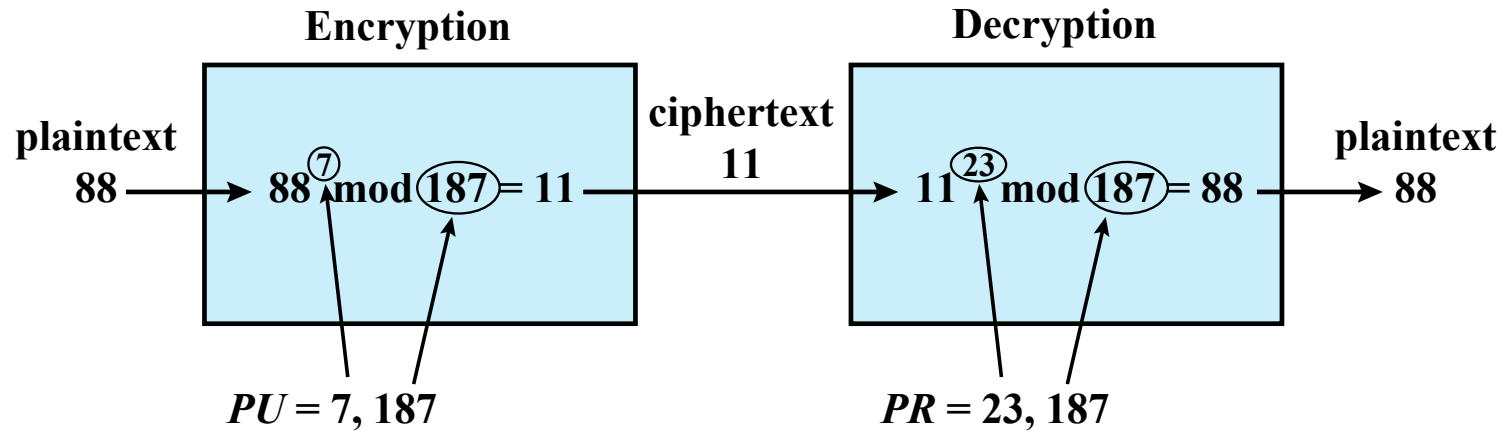
Decryption

Ciphertext:	C
Plaintext:	$M = C^d \pmod{n}$

$(d \cdot e) \bmod \phi(n) = 1$ (it implies $d \cdot e = k \cdot \phi(n) + 1$, where k is an integer).

$d = (k \cdot \phi(n) + 1) / e$, where k is an integer

Example of RSA Algorithm



Public Key is the pair (e, n)
Private Key is the pair (d, n)

1. Select two prime numbers, $p = 17$ and $q = 11$.
2. Calculate $n = pq = 17 \times 11 = 187$.
3. Calculate $\phi(n) = (p - 1)(q - 1) = 16 \times 10 = 160$.
4. Select e so that e is relatively prime to $\phi(n) = 160$ and less than $\phi(n)$; we choose $e = 7$.
5. Determine d such that $de \bmod 160 = 1$ and $d < 160$. The correct value is $d = 23$, because $23 \times 7 = 161 = (1 \times 160) + 1$.

*Note that $d = (k * \phi(n) + 1) / e$; both k and d are integers. Thus, when k is 1, d is 23.

*If you can know $\phi(n)$, you can significantly narrow down the number of possible private keys !!!

RSA Factoring Challenge

Encryption	
Plaintext:	$M < n$
Ciphertext:	$C = M^e \pmod n$

Decryption	
Ciphertext:	C
Plaintext:	$M = C^d \pmod n$

Key Generation	
Select p, q	p and q both prime, $p \neq q$
Calculate $n = p \cdot q$	
Calculate $\phi(n) = (p-1)(q-1)$	
Select integer e	$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$
Calculate d	$de \pmod{\phi(n)} = 1$
Public key	$KU = \{e, n\}$
Private key	$KR = \{d, n\}$

- For an adversary, he/she can get public key $\{e$ and $n\}$ and C (ciphertext).
- To decipher C , the adversary needs to infer private key d so that he/she can infer M (plaintext) = $C^d \pmod n$
- How to discover d (private key)?
 - $d = (k \cdot \phi(n) + 1) / e$, where k is an integer
 - There is no efficient solution to calculate $\phi(n)$. The best algorithm requires $O(\sqrt{n})$ if you cannot factorize n first (recall: $\phi(n) = (p-1) \cdot (q-1)$).

Question: How to encrypt data by RSA?

E.g., encrypting “abc”

Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex
(sp)	32	0040	0x20	@	64	0100	0x40	`	96	0140	0x60
!	33	0041	0x21	A	65	0101	0x41	a	97	0141	0x61
"	34	0042	0x22	B	66	0102	0x42	b	98	0142	0x62
#	35	0043	0x23	C	67	0103	0x43	c	99	0143	0x63
\$	36	0044	0x24	D	68	0104	0x44	d	100	0144	0x64
%	37	0045	0x25	E	69	0105	0x45	e	101	0145	0x65
&	38	0046	0x26	F	70	0106	0x46	f	102	0146	0x66
'	39	0047	0x27	G	71	0107	0x47	g	103	0147	0x67
(	40	0050	0x28	H	72	0110	0x48	h	104	0150	0x68
)	41	0051	0x29	I	73	0111	0x49	i	105	0151	0x69
*	42	0052	0x2a	J	74	0112	0x4a	j	106	0152	0x6a
+	43	0053	0x2b	K	75	0113	0x4b	k	107	0153	0x6b
,	44	0054	0x2c	L	76	0114	0x4c	l	108	0154	0x6c
-	45	0055	0x2d	M	77	0115	0x4d	m	109	0155	0x6d
.	46	0056	0x2e	N	78	0116	0x4e	n	110	0156	0x6e
/	47	0057	0x2f	O	79	0117	0x4f	o	111	0157	0x6f
0	48	0060	0x30	P	80	0120	0x50	p	112	0160	0x70
1	49	0061	0x31	Q	81	0121	0x51	q	113	0161	0x71
2	50	0062	0x32	R	82	0122	0x52	r	114	0162	0x72
3	51	0063	0x33	S	83	0123	0x53	s	115	0163	0x73
4	52	0064	0x34	T	84	0124	0x54	t	116	0164	0x74
5	53	0065	0x35	U	85	0125	0x55	u	117	0165	0x75
6	54	0066	0x36	V	86	0126	0x56	v	118	0166	0x76
7	55	0067	0x37	W	87	0127	0x57	w	119	0167	0x77
8	56	0070	0x38	X	88	0130	0x58	x	120	0170	0x78
9	57	0071	0x39	Y	89	0131	0x59	y	121	0171	0x79
:	58	0072	0x3a	Z	90	0132	0x5a	z	122	0172	0x7a
;	59	0073	0x3b	[	91	0133	0x5b	{	123	0173	0x7b
<	60	0074	0x3c	\	92	0134	0x5c		124	0174	0x7c
=	61	0075	0x3d	]	93	0135	0x5d	}	125	0175	0x7d
>	62	0076	0x3e	^	94	0136	0x5e	~	126	0176	0x7e
?	63	0077	0x3f	_	95	0137	0x5f				

Question: How do you encrypt the string “abc” using ASCII encoding and the public key $(e, n) = (7, 187)$?

Answer: (1) Convert each character to ASCII: $a = 97, b = 98, c = 99$.

(2) For each block compute $C = M^7 \bmod 187$ using the public key $(7, 187)$.

$$97^7 \bmod 187 = 92$$

(3) Send the ciphertext blocks in order.

Actual values: $97 \rightarrow 92, 98 \rightarrow 21, 99 \rightarrow 176$.

Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex
(sp)	32	0040	0x20	@	64	0100	0x40	`	96	0140	0x60
!	33	0041	0x21	A	65	0101	0x41	a	97	0141	0x61
"	34	0042	0x22	B	66	0102	0x42	b	98	0142	0x62
#	35	0043	0x23	C	67	0103	0x43	c	99	0143	0x63
\$	36	0044	0x24	D	68	0104	0x44	d	100	0144	0x64
%	37	0045	0x25	E	69	0105	0x45	e	101	0145	0x65
&	38	0046	0x26	F	70	0106	0x46	f	102	0146	0x66
'	39	0047	0x27	G	71	0107	0x47	g	103	0147	0x67
(	40	0050	0x28	H	72	0110	0x48	h	104	0150	0x68
)	41	0051	0x29	I	73	0111	0x49	i	105	0151	0x69
*	42	0052	0x2a	J	74	0112	0x4a	j	106	0152	0x6a
+	43	0053	0x2b	K	75	0113	0x4b	k	107	0153	0x6b
,	44	0054	0x2c	L	76	0114	0x4c	l	108	0154	0x6c
-	45	0055	0x2d	M	77	0115	0x4d	m	109	0155	0x6d
.	46	0056	0x2e	N	78	0116	0x4e	n	110	0156	0x6e
/	47	0057	0x2f	O	79	0117	0x4f	o	111	0157	0x6f
0	48	0060	0x30	P	80	0120	0x50	p	112	0160	0x70
1	49	0061	0x31	Q	81	0121	0x51	q	113	0161	0x71
2	50	0062	0x32	R	82	0122	0x52	r	114	0162	0x72
3	51	0063	0x33	S	83	0123	0x53	s	115	0163	0x73
4	52	0064	0x34	T	84	0124	0x54	t	116	0164	0x74
5	53	0065	0x35	U	85	0125	0x55	u	117	0165	0x75
6	54	0066	0x36	V	86	0126	0x56	v	118	0166	0x76
7	55	0067	0x37	W	87	0127	0x57	w	119	0167	0x77
8	56	0070	0x38	X	88	0130	0x58	x	120	0170	0x78
9	57	0071	0x39	Y	89	0131	0x59	y	121	0171	0x79
:	58	0072	0x3a	Z	90	0132	0x5a	z	122	0172	0x7a
;	59	0073	0x3b	[	91	0133	0x5b	{	123	0173	0x7b
<	60	0074	0x3c	\	92	0134	0x5c		124	0174	0x7c
=	61	0075	0x3d	]	93	0135	0x5d	}	125	0175	0x7d
>	62	0076	0x3e	^	94	0136	0x5e	~	126	0176	0x7e
?	63	0077	0x3f	_	95	0137	0x5f				

Common Asymmetric Algorithms

- **RSA:** The most widely used asymmetric algorithm. Great for encrypting small chunks of data and creating digital signatures
- **ECC (Elliptic Curve Cryptography):** Newer than RSA, offers similar security with shorter key lengths - perfect for mobile devices and IoT
- **Diffie-Hellman:** It is not exactly encryption, but it uses asymmetric math to help two parties create a shared secret key

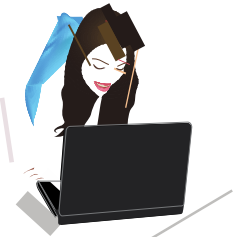
Diffie-Hellman Key Exchange Algorithm

- The Diffie-Hellman Key Exchange is one of the most significant inventions in cybersecurity, proposed by Whitfield Diffie and Martin Hellman in 1976
- Diffie-Hellman (DH) is an **asymmetric algorithm** used to create a **symmetric key**
 - **It is an Asymmetric Process:** Each party has a private key (a secret number) and a public key (a calculated value shared openly)
 - **Its goal is to get a Symmetric Key:** It enables two entities to agree upon a secret key without prior arrangements, even in the presence of potential eavesdroppers (i.e., over an insecure channel)

Why Not Use Asymmetric Encryption for Everything?

- **Asymmetric (RSA) is Heavy:** It relies on complex modular exponentiation of massive prime numbers. This is mathematically "expensive" and slow
- **Symmetric (AES) is Fast:** It uses simple bitwise shuffling and substitutions that modern CPUs can process almost instantly
- Modern HTTPS (specifically using **TLS 1.3**) acts as a "Hybrid" system, combining the strengths of **Asymmetric** and **Symmetric** cryptography
 - Step 1: The Handshake (Identity & Negotiation, RSA/Digital Signatures)
 - Step 2: Key Exchange (Diffie-Hellman)
 - Step 3: Bulk Encryption (Symmetric AES)

Diffie-Hellman Key Exchange



Alice



Bob

Alice and Bob share a prime q and α , such that $\alpha < q$ and α is a primitive root of q

Alice and Bob share a prime q and α , such that $\alpha < q$ and α is a primitive root of q

Alice generates a private key X_A such that $X_A < q$

Bob generates a private key X_B such that $X_B < q$

Alice calculates a public key $Y_A = \alpha^{X_A} \bmod q$

Bob calculates a public key $Y_B = \alpha^{X_B} \bmod q$

Alice receives Bob's public key Y_B in plaintext

Bob receives Alice's public key Y_A in plaintext

Alice calculates shared secret key $K = (Y_B)^{X_A} \bmod q$

Bob calculates shared secret key $K = (Y_A)^{X_B} \bmod q$



1. Parameters Setup

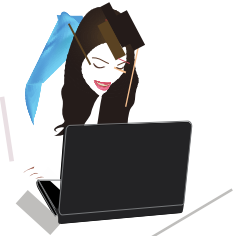
Alice and Bob must agree upon two number:

- A large prime number q ,
- A generator α , which is the primitive root of q

For example, if

- Prime number $q = 353$
- Generator $\alpha = 3$

Diffie-Hellman Key Exchange



Alice



Bob

Alice and Bob share a prime q and α , such that $\alpha < q$ and α is a primitive root of q

Alice and Bob share a prime q and α , such that $\alpha < q$ and α is a primitive root of q

Alice generates a private key X_A such that $X_A < q$

Bob generates a private key X_B such that $X_B < q$

Alice calculates a public key $Y_A = \alpha^{X_A} \bmod q$

Bob calculates a public key $Y_B = \alpha^{X_B} \bmod q$

Alice receives Bob's public key Y_B in plaintext

Bob receives Alice's public key Y_A in plaintext

Alice calculates shared secret key $K = (Y_B)^{X_A} \bmod q$

Bob calculates shared secret key $K = (Y_A)^{X_B} \bmod q$



2. Private Key Generation

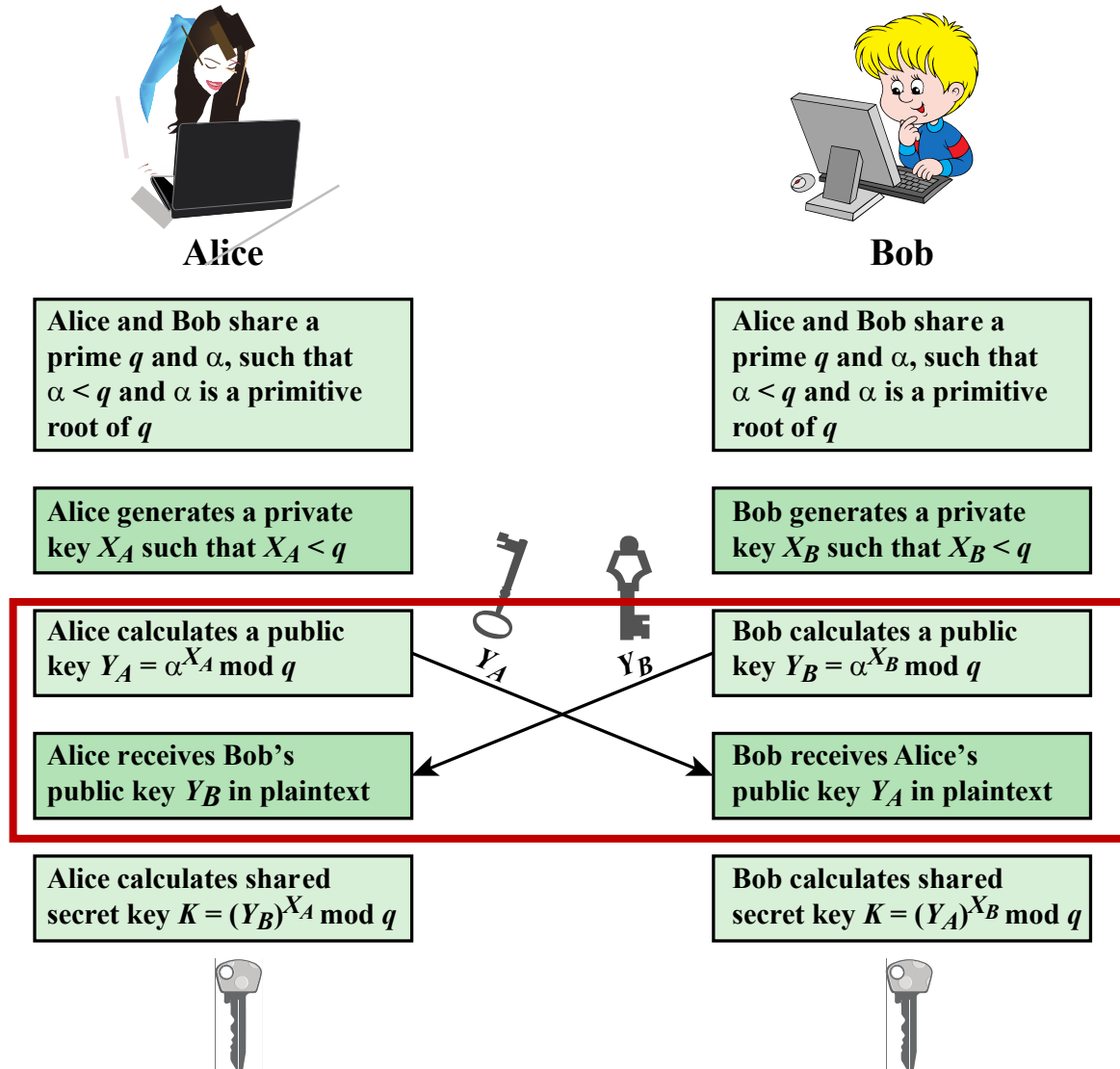
Alice and Bob each randomly choose a large integer to serve as their private key.

These private keys are kept secret and not being shared

For example,

- A selects her private key X_A of 97
- B selects his private key X_B of 233

Diffie-Hellman Key Exchange



3. Public Key Exchange

Both Alice and Bob perform a calculation to generate their corresponding public keys

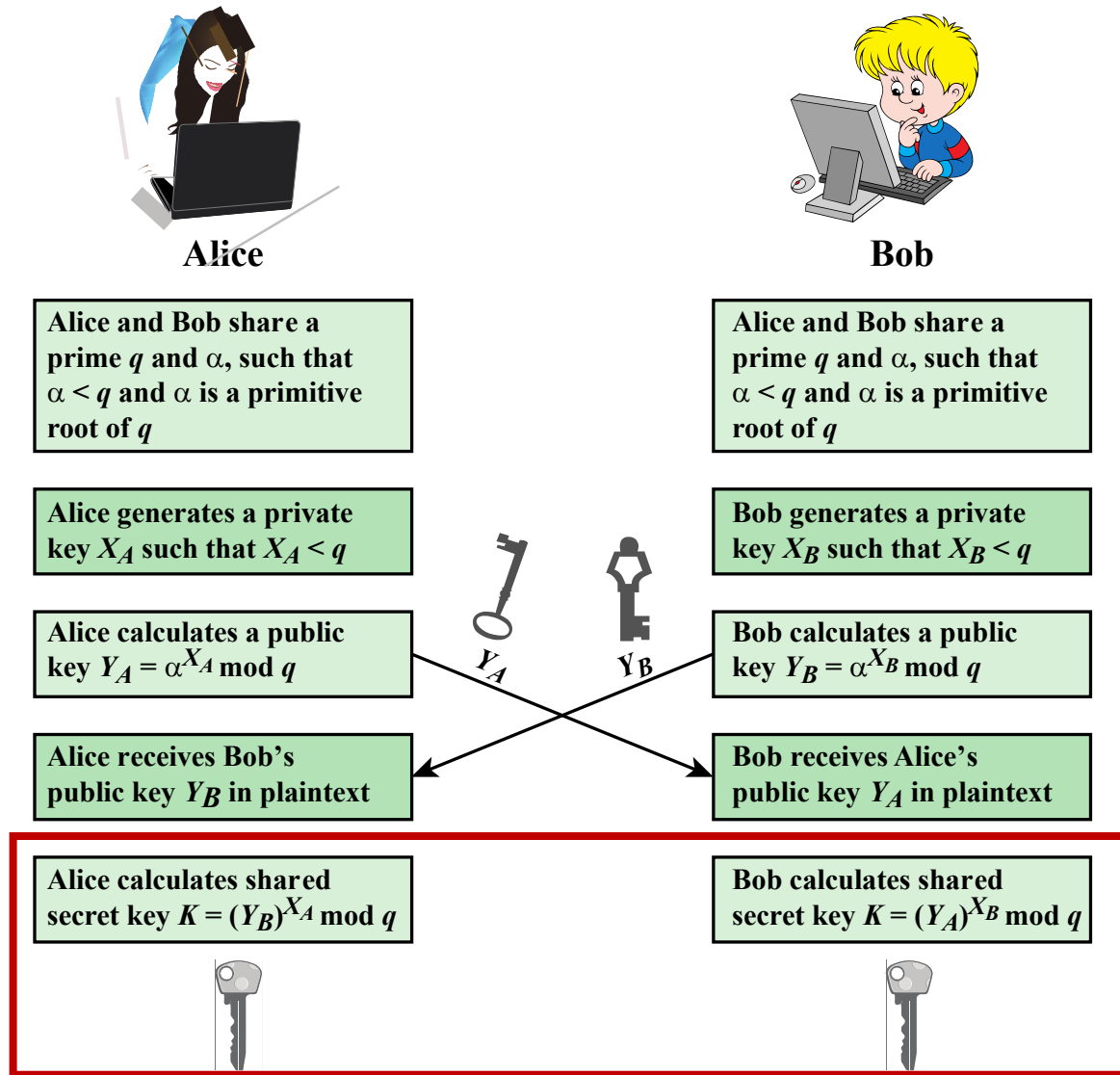
- The public key of Alice: $Y_A = \alpha^{X_A} \bmod q$
- The public key of Bob: $Y_B = \alpha^{X_B} \bmod q$

The public key are then shared with each other

In our examples,

- Public key of Alice: $Y_A = 3^{97} \bmod 353 = 40$
- Public key of Bob: $Y_B = 3^{233} \bmod 353 = 248$

Diffie-Hellman Key Exchange



3. Shared Secret Key Calculation

Alice then calculates the shared secret using the Y_B received from Bob and her private key as:

$$k = (Y_B)^{X_A} \pmod{q}$$

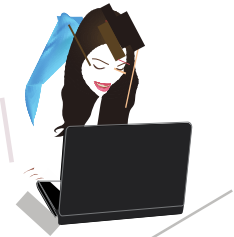
Bob also calculates the shared secret using the Y_A received from Alice and his private key as:

$$k = (Y_A)^{X_B} \pmod{q}$$

In our examples,

$$K = 248^{97} \bmod 353 = 40^{233} \bmod 353 = 160$$

Diffie-Hellman Key Exchange

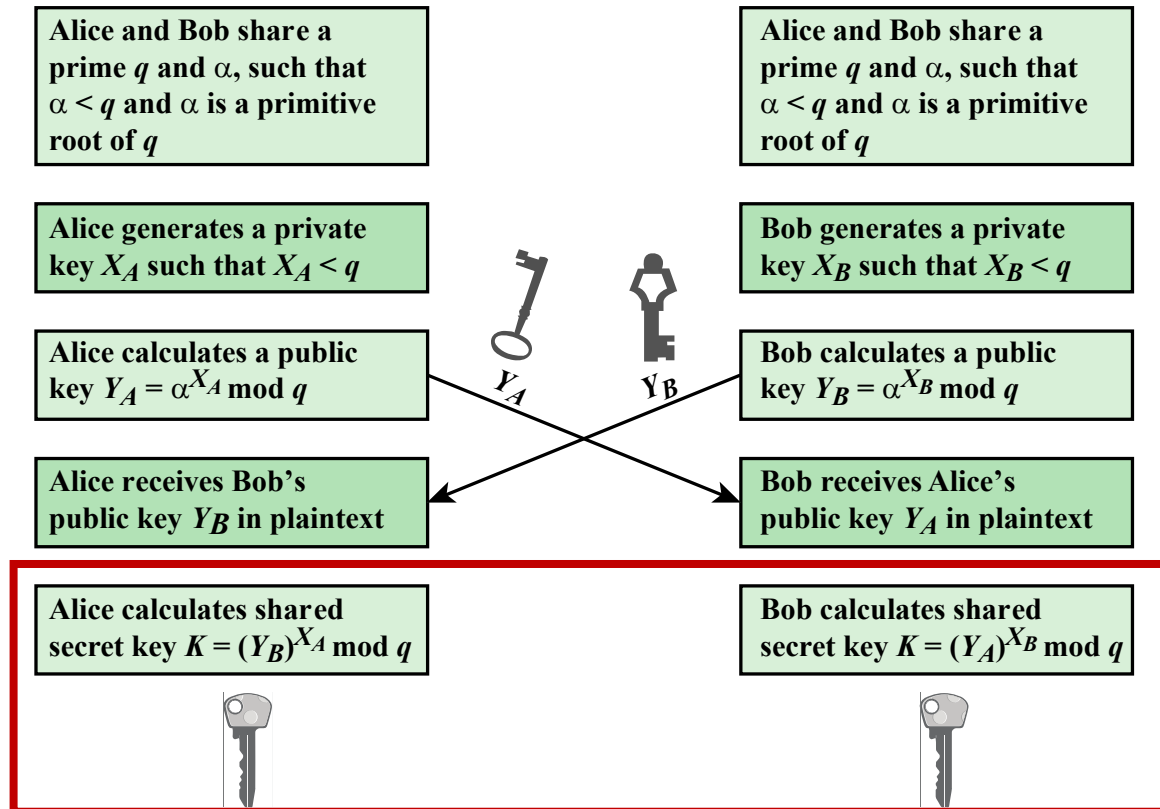


Alice



Bob

Why Alice and Bob can get the same secret key K ?



- $k = (Y_B)^{X_A} \bmod q = (\alpha^{X_B} \bmod q)^{X_A} \bmod q = \alpha^{X_B * X_A} \bmod q$
- $k = (Y_A)^{X_B} \bmod q = (\alpha^{X_A} \bmod q)^{X_B} \bmod q = \alpha^{X_B * X_A} \bmod q$

Man-in-the-Middle Attack

- Attack is:

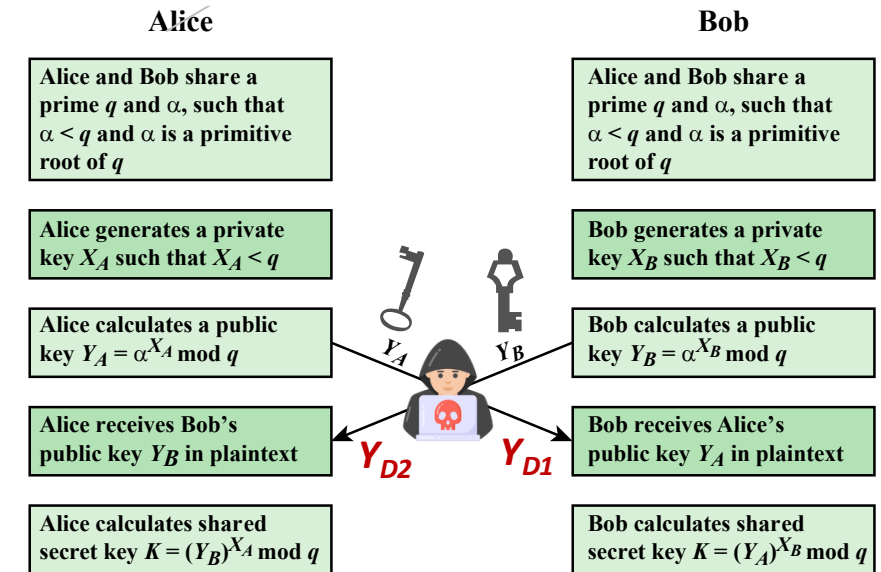
1. Darth generates two secret keys X_{D1} and X_{D2} , and calculates the corresponding public values:

$$Y_{D1} = (\alpha^{X_{D1}}) \bmod q$$

$$Y_{D2} = (\alpha^{X_{D2}}) \bmod q$$

2. Alice transmits her public value Y_A to Bob.
3. Darth **intercepts** Y_A and transmits his own Y_{D1} to Bob (pretending to be Alice).
Darth also calculates the key $K2$ for communicating with Alice:

$$K2 = (Y_A^{X_{D2}}) \bmod q$$



Man-in-the-Middle Attack

4. Bob receives Y_{D1} (mistakenly believing it is from Alice) and calculates the key $K1$:

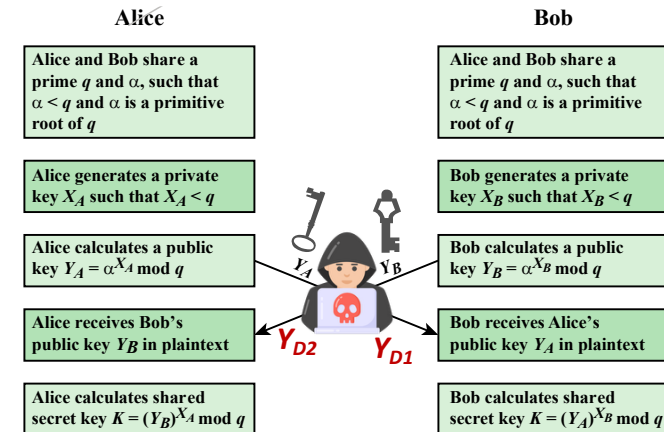
$$K1 = (Y_{D1}^{X_B}) \bmod q$$

5. Bob transmits his public value Y_B to Alice.
6. Darth **intercepts** Y_B and transmits his own Y_{D2} to Alice (pretending to be Bob).
Darth calculates the key $K1$ for communicating with Bob:

$$K1 = (Y_B^{X_{D1}}) \bmod q$$

7. Alice thinks she is communicating with Bob using $K2$, and Bob thinks he is communicating with Alice using $K1$.

In reality, both are communicating with the man-in-the-middle, Darth. Since Darth holds both $K1$ and $K2$, he can decrypt, read, alter, and forward all messages between them.



Man-in-the-Middle Attack

4. Bob receives Y_{D1} (mistakenly believing it is from Alice) and calculates the key $K1$:

$$K1 = (Y_{D1}^{X_B}) \bmod q$$

5. Bob transmits his public value Y_B to Alice.
6. Darth **intercepts** Y_B and transmits his own Y_{D2} to Alice (pretending to be Bob).
Darth calculates the key $K1$ for communicating with Bob:

$$K1 = (Y_B^{X_{D1}}) \bmod q$$

7. Alice receives Y_{D2} and calculates the key $K2$:

**Any defense against DH Key Exchange
Man-in-the-Middle Attack?**

How TLS combines RSA and Diffie-Hellman

- **UMBC server → Alice: certificate**, containing UMBC's RSA public key (e, n) , signed by a CA. Alice checks the CA against her root store.
→ *Alice now trusts that (e, n) is UMBC's public key.*
- **Alice → UMBC server: Alice's DH public value Y_A** ($= \alpha^{X_A} \bmod q$).
→ *UMBC computes $K = Y_A^{X_B} \bmod q$.*
- **UMBC server → Alice: its DH public value Y_B** ($= \alpha^{X_B} \bmod q$) + **a signature on Y_B** , made with UMBC's RSA private key (d, n) . **Alice verifies the signature with (e, n)** . Valid → Y_B really came from UMBC, not from a man-in-the-middle. → *Alice computes $K = Y_B^{X_A} \bmod q$.*
- **Both sides now hold the same shared secret K .** X_A and X_B never left their machines.
- **All further communication is encrypted with a symmetric cipher (AES) keyed from K .** RSA and DH are not used again in this connection.

How TLS combines RSA and Diffie-Hellman

Why doesn't Alice send her RSA public key?

- **She has no certificate.** Public CAs vouch for *domain names* (umbc.edu), which they can verify.
- **TLS therefore authenticates one direction by default.** The server proves who it is. Anyone — including an attacker — can open an HTTPS connection to umbc.edu; the user proves who *they* are later, *inside* the encrypted channel, when it matters (e.g., accessing a dataset): UMBC ID, password, 2FA.
- **The asymmetry is enough.** The attacker can't impersonate UMBC to Alice (the signature check in step 4). Impersonating Alice to UMBC gains him nothing — he gets an encrypted channel to a login page, same as anyone else.